

Document Version: 5.0 · Final Master Release

Platform Type: Serverless Cloud-Native E-Commerce Web Application

Frontend Stack: HTML5 · Vanilla CSS3 · Modular ES6 JavaScript

Backend / DB: Google Firebase Auth · Cloud Firestore NoSQL

Hosting: Firebase Hosting (Project: `nari-niketan`) + AWS CDN Option

Security Model: Server-Enforced RBAC · Firestore Declarative Rules

NARI NIKETAN

MASTER SYSTEM ANALYSIS · ARCHITECTURE · TECHNOLOGY BLUEPRINT · IMPLEMENTATION GUIDE

1. EXECUTIVE SUMMARY & BUSINESS OBJECTIVES

Nari Niketan (*Sanskrit: Women's Sanctuary*) is a full-featured, production-grade E-Commerce web application serving the Indian ethnic fashion market — specialising in Sarees, Suits, Lehengas, Kurtas, and Dupattas. The platform was architected as a completely serverless, zero-infrastructure application, meaning **no dedicated backend servers, no Linux VMs, and no DevOps overhead** are required to operate it at global scale.

Core Design Philosophy

Every architecture decision in Nari Niketan was made to maximise **speed, security, scalability, and operational simplicity**. The result is a platform that can handle 0 to 100,000+ concurrent shoppers without provisioning a single server, patching a single operating system, or managing a single database backup.

CAPABILITY AREA	CUSTOMER FEATURES	ADMINISTRATOR FEATURES
Catalog & Products	Multi-category filtering, price slider, size/color chips, multi-image gallery, stock badge, ratings	Full Product CRUD (Add/Edit/Delete), image URL management, pricing, stock controls
Authentication & Identity	Email/Password, 1-Click Google OAuth 2.0, SMS Phone OTP + reCAPTCHA	Dedicated admin gate, Master Key activation (<code>NARINIKETAN2024</code>), <code>isAdmin</code> role verification
Shopping Cart & Coupons	LocalStorage cart engine, variant selection (size, color, qty), live promo code validator (<code>NARI20</code> , <code>WELCOME10</code>), GST breakdown	Coupon CRUD: percentage & flat discounts, minimum order thresholds, activation toggles
Order Lifecycle	Checkout (COD/UPI/Card), order confirmation receipt, real-time status tracker, 1-Click Self-Service Cancellation	Revenue dashboard, live order table, fulfillment stage updater (Pending → Shipped → Delivered), cancellation logs

2. WEBSITE ARCHITECTURE DIAGRAM — PAGE NAVIGATION & USER FLOW

The diagram below shows **every page** in the Nari Niketan website, how they connect to each other, and which user role can access them. This is the recommended *site architecture* view that maps the complete user journey from landing to order completion.

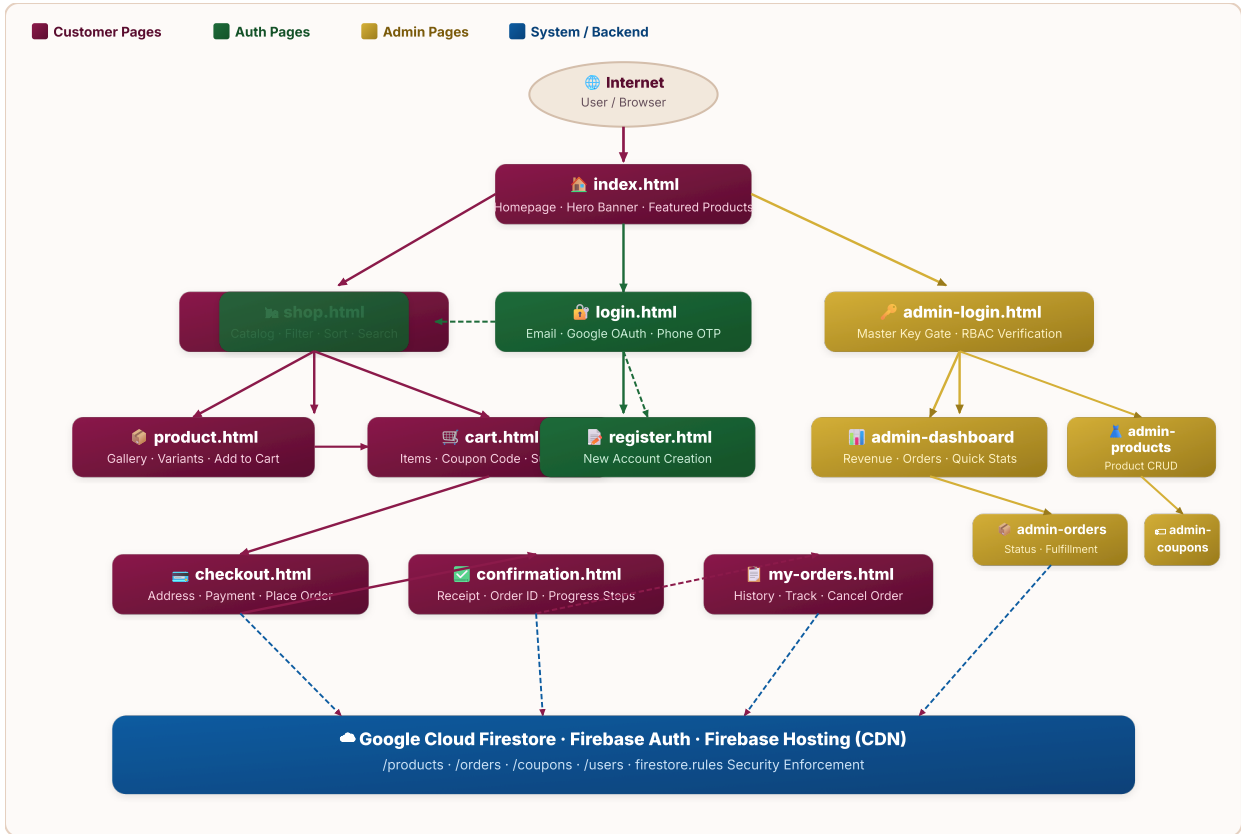


FIGURE 1 — WEBSITE PAGE ARCHITECTURE, NAVIGATION FLOW & USER JOURNEY MAP (CUSTOMER + ADMIN PATHS)

Reading the diagram: Burgundy/red nodes = Customer-facing pages. Green nodes = Authentication pages. Gold nodes = Admin-only pages. Blue = Cloud backend. Solid arrows show primary navigation flows; dashed arrows show conditional or optional transitions. All pages connect to Cloud Firestore for live data operations.

3. SYSTEM ARCHITECTURE — MULTI-TIER TECHNICAL TOPOLOGY

The following diagram shows the **technical architecture**: how the browser communicates with Firebase's global CDN, how authentication tokens flow, and how Firestore security rules gate every data operation.

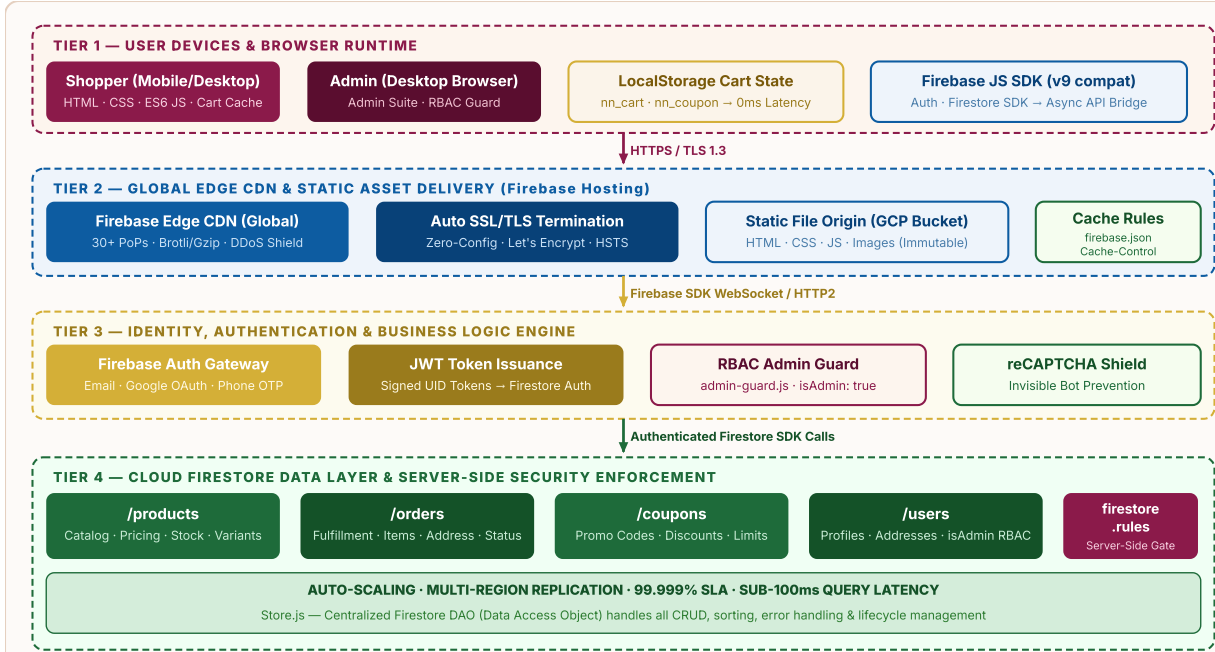


FIGURE 2 — FOUR-TIER TECHNICAL SYSTEM ARCHITECTURE: BROWSER → CDN → AUTH → CLOUD FIRESTORE

4. TECHNOLOGY STACK & WHY EACH WAS CHOSEN OVER ALTERNATIVES

Every technology choice was validated against real-world performance requirements, cost constraints, security standards, and long-term maintainability for an Indian ethnic fashion E-Commerce platform serving mobile-first users.

4.1 Frontend: Semantic HTML5 + Vanilla CSS3 + Vanilla ES6 JavaScript

✓ CHOSEN

✓ Chosen: Vanilla HTML5 / CSS3 / ES6

- **Oms hydration overhead** — page renders immediately without JavaScript framework bootstrap
- **No npm / build toolchain** — no Webpack, Vite, Babel, or dependency version conflicts
- **3G/4G mobile optimised** — ultra-small payload critical for Indian tier-2/3 city shoppers
- **No breaking updates** — native web standards last decades; React 18→19 broke many apps
- **Direct Firebase SDK integration** — no additional adapter layers needed

✗ Why Not React / Next.js / Angular / Vue?

- **300–600KB+ runtime bundle** just to show a product list, catastrophic on slow networks
- **Server cost** — Next.js SSR requires always-on Node.js server or expensive edge workers
- **Build pipeline complexity** — npm security vulnerabilities, Babel transpiler updates
- **Overkill for scope** — for a content-driven storefront, the framework adds zero user value

4.2 Styling: Bespoke Vanilla CSS3 Design System

✓ CHOSEN

✓ Chosen: Custom CSS3 with CSS Variables

- **Authentic Indian aesthetic** — Burgundy `#8B1A4A` & Gold `#D4AF37` impossible to achieve naturally with utility-class systems
- **Zero unused CSS** — no Tailwind purge complexity, every line serves a purpose
- **Full design control** — custom micro-animations, glassmorphism, gradient overlays
- **Smaller file size** — 4 purpose-built CSS files vs. a 30KB Tailwind base reset

✗ Why Not Tailwind CSS / Bootstrap / Material UI?

- **Tailwind** — generic look; ethnic artisanal brand needs custom personality, not utility classes
- **Bootstrap** — opinionated components clash with bespoke Indian royal design language
- **Material UI** — Google's flat design aesthetic conflicts with Nari Niketan's luxury positioning

4.3 Database: Google Cloud Firestore (NoSQL Document Store)

✓ CHOSEN

✓ Chosen: Cloud Firestore

- **Direct browser access** — no Express/Node.js API server middleware needed
- **Sub-100ms multi-region reads** — product catalog feels instant to shoppers
- **Serverless, zero provisioning** — scales from 0 to 100k+ users with no config change
- **Declarative `firestore.rules`** — server-enforced security without writing a single API endpoint
- **Free tier** — 50k reads/day free, perfect for startup launch phase
- **Offline resilience** — SDK caches data locally, works even on flaky 2G connections

✗ Why Not PostgreSQL / MySQL / MongoDB?

- **SQL databases** — need a dedicated EC2/RDS server (₹3,000–₹15,000/month), OS patching, connection pools
- **Require backend API** — browsers cannot connect to SQL databases directly; need Express/Django/Rails server
- **MongoDB Atlas** — free tier is limited; direct browser access requires Atlas Data API with significant latency overhead
- **No built-in security rules** — all access control must be hand-coded in API layer

4.4 Authentication: Google Firebase Authentication

✓ CHOSEN

✓ Chosen: Firebase Auth

- **3-in-1 auth out of the box** — Email/Password, Google OAuth, Phone SMS OTP in one SDK
- **JWT tokens auto-integrate with Firestore rules** — `request.auth.uid` available in every security rule
- **Invisible reCAPTCHA** — stops SMS OTP abuse and credential stuffing at zero integration cost
- **SOC 2 / GDPR compliant** — Google manages all password hashing, credential storage liability
- **Zero cost up to 10k MAU** — no billing for authentication at startup scale

✗ Why Not Custom JWT / Auth0 / Passport.js?

- **Custom JWT** — must manage bcrypt salting, token rotation, refresh tokens, XSS cookie security, SMS gateway API keys (Twilio ₹4,000/month)
- **Auth0** — \$240+/month for production MAU, plus complex callback URL management
- **Passport.js** — requires a Node.js backend server to run, eliminates serverless architecture

4.5 Cart State: Client-Side LocalStorage Engine

✓ CHOSEN

✓ Chosen: Synchronous LocalStorage (`nn_cart` key)

✗ Why Not Server-Side Sessions / Redis?

- **Truly instant updates** — `localStorage.setItem()` is synchronous, 0ms network delay
- **Works before login** — guest users build a full cart without creating an account
- **Survives page navigation** — cart persists across browsing session without server calls
- **Coupon state cached** — `nn_coupon` key stores applied discount across page loads

- **Redis** — requires a running server (₹1,500–₹5,000/month on ElastiCache), defeats serverless philosophy
- **Database-backed sessions** — every "Add to Cart" click becomes a Firestore write, generating cost & latency
- **Session expiry complexity** — abandoned carts pollute the database without costly TTL cleanup workers

4.6 Hosting: Firebase Hosting (Primary)

✓ CHOSEN

✓ Chosen: Firebase Hosting (Project: `nari-niketan`)

- **URLs:** `nari-niketan.web.app` / `nari-niketan.firebaseio.com`
- **30+ global PoPs** — edge caches serve pages from the nearest server to the shopper
- **Auto SSL/TLS** — HTTPS enabled with zero configuration, auto-renews certificates
- **One-command deploy:** `firebase deploy` pushes code + Firestore rules simultaneously
- **Free tier** — 10GB storage, 360MB/day transfer before any billing
- **Custom domain support** — connect a `.in` domain with DNS verification

✗ Why Not AWS EC2 / cPanel / Vercel / Netlify?

- **EC2** — requires Linux administration, OS patching, NGINX configuration, SSL setup, backups
- **cPanel shared hosting** — no WebSocket support, limited CDN, poor global performance
- **Vercel/Netlify** — viable alternatives, but Firebase Hosting + Firestore rules deploy together atomically as one unit, maintaining consistency

5. DATA FLOW & ORDER LIFECYCLE SEQUENCE DIAGRAM

This diagram traces the exact data exchange sequence from a shopper clicking "Add to Cart" through to order placement in Firestore, admin fulfillment, and 1-Click cancellation:

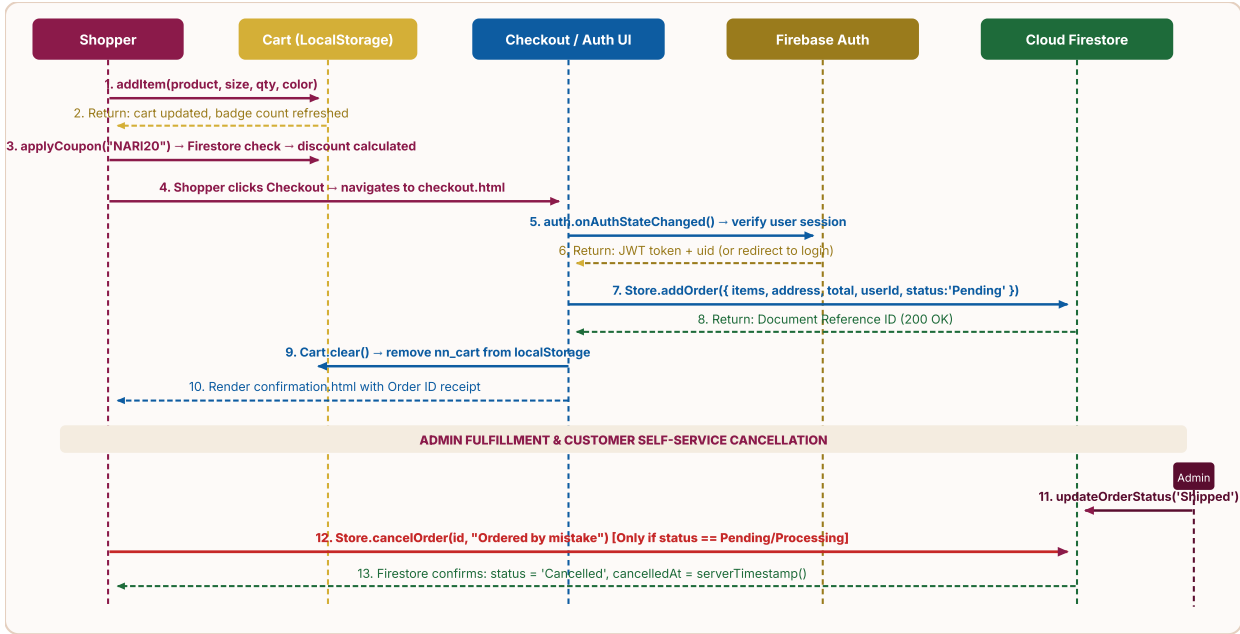


FIGURE 3 — END-TO-END DATA FLOW: ADD TO CART → CHECKOUT → ORDER PLACEMENT → FULFILLMENT → CANCELLATION

6. DATABASE SCHEMA & FIRESTORE ENTITY-RELATIONSHIP MODEL

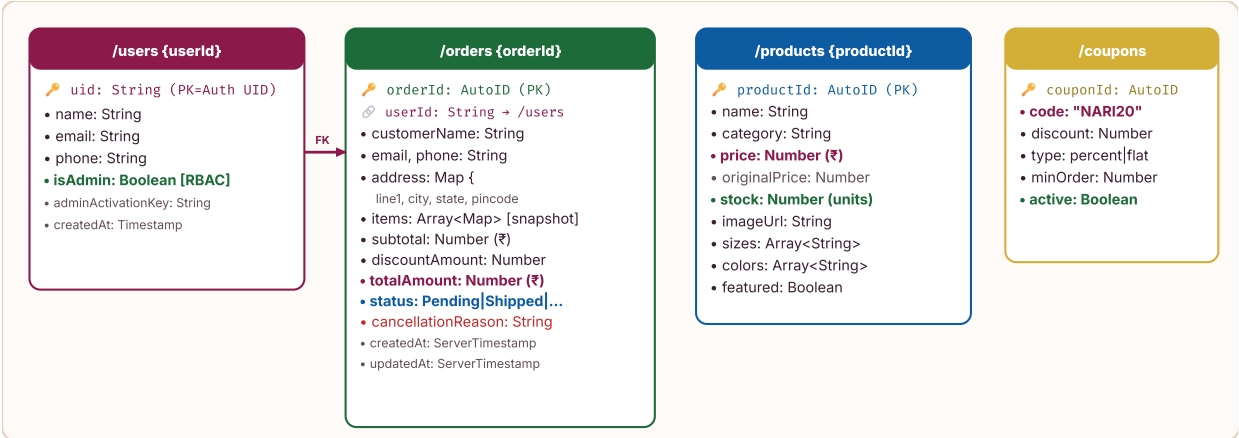


FIGURE 4 — CLOUD FIRESTORE DOCUMENT COLLECTIONS ENTITY-RELATIONSHIP (ER) SCHEMA

COLLECTION	KEY FIELDS	ACCESS PATTERN	WHO CAN WRITE
/products	name, category, price, stock, sizes, colors, imageUrl	Public reads; category filter; featured query	Admin only (isAdmin = true)
/orders	userId, items[], status, totalAmount, address{}	User reads own orders; Admin reads all; filter by status	Customer creates own; customer cancels own; Admin full control
/coupons	code, discount, type, minOrder, active	Public reads; code lookup by exact match	Admin only
/users	isAdmin, name, email, adminActivationKey	Users read/write own profile	User creates own; isAdmin protected from self-elevation

7. AUTHENTICATION & RBAC SECURITY ENGINE

7.1 Multi-Provider Authentication Flow

1 Email / Password

User registers via `register.html`. Firebase creates a credential record. On login, Firebase validates and issues a signed JWT. `Store.createUserProfile(uid, { ... })` writes a `/users/{uid}` document with `isAdmin: false`.

2 Google OAuth 2.0 (1-Click Sign-In)

User clicks "Sign in with Google" popup via `signInWithPopup(googleProvider)`. Google authenticates and returns an ID token. Firebase Auth exchanges it for a Firebase JWT, available as `auth.currentUser`.

3 SMS Phone OTP + reCAPTCHA

User enters phone number. `auth.signInWithPhoneNumber()` triggers an SMS OTP via Google's telephony network. Invisible reCAPTCHA validates the browser is human before sending SMS. OTP verification completes sign-in.

4 Admin Role Activation

Admin visits `admin-login.html`, enters email + Master Key `NARINIKETAN2024`. `Store.setAdminRole(uid)` sets `isAdmin: true` in Firestore. `admin-guard.js` verifies this flag on every admin page load.

7.2 Firestore Security Rules (Production)

```
rules_version = '2';
service cloud.firestore {
  match /databases/{database}/documents {

    function isAuthenticated() { return request.auth != null; }

    // Server-side admin check – reads from /users collection
    function isAdmin() {
      return isAuthenticated() &&
        exists(/databases/{database}/documents/users/{request.auth.uid}) &&
        get(/databases/{database}/documents/users/{request.auth.uid}).data.isAdmin
    }

    // /products – Public catalog read; Admin-only write
    match /products/{id} {
      allow read: if true;
      allow write, create, update, delete: if isAdmin();
    }

    // /orders – Customer creates/reads own; Customer can ONLY set Cancelled; Admin f
    match /orders/{id} {
```

```

allow create: if isAuthenticated() && request.resource.data.userId = request.a
allow read:   if isAuthenticated() && (resource.data.userId = request.auth.uid
allow update: if isAuthenticated() && (
    isAdmin() ||
    (resource.data.userId = request.auth.uid && request.resource.data.status =
);
allow delete: if isAdmin();
}

// /coupons - Public read; Admin-only write
match /coupons/{id} {
    allow read: if true;
    allow write, create, update, delete: if isAdmin();
}

// /users - Own profile read/write; isAdmin cannot be self-elevated (except with
match /users/{userId} {
    allow read:   if isAuthenticated() && (request.auth.uid = userId || isAdmin())
    allow create: if isAuthenticated() && request.auth.uid = userId;
    allow update: if isAuthenticated() && (
        isAdmin() ||
        (request.auth.uid = userId &&
            !request.resource.data.diff(resource.data).affectedKeys().hasAny(['isAdmin'
        (request.auth.uid = userId && request.resource.data.adminActivationKey = "N
    );
    allow delete: if isAdmin();
}
}
}

```

8. COMPLETE FILE HIERARCHY & COMPONENT MAP

FILE	ROLE	KEY RESPONSIBILITIES
index.html	Storefront	Homepage: Hero banner, promotional countdowns, featured product grid (calls <code>Store.getProducts({featured:true})</code>)
shop.html	Storefront	Full catalog: real-time client-side filters for category, price slider, size/color chips, sort options
product.html	Storefront	Product detail: multi-image gallery, variant picker (size, color, qty), stock badge, "Add to Cart" / "Buy Now"
cart.html	Storefront	Cart drawer: reads <code>nn_cart</code> from localStorage, quantity controls, coupon code validator, GST/discount breakdown
checkout.html	Storefront	Checkout: delivery address form, payment selector (COD/UPI/Card), calls <code>Store.addOrder()</code> , clears cart on success
confirmation.html	Storefront	Success page: order ID receipt, animated progress steps, links to <code>my-orders.html</code>
my-orders.html	Storefront	Orders dashboard: calls <code>Store.getUserOrders(uid)</code> , real-time status badges, 1-Click Cancellation modal with reason selector
login.html	Auth	Unified login: Email/Password, Google OAuth popup, Phone OTP (reCAPTCHA widget render)
register.html	Auth	Registration form with inline validation; calls <code>createUserWithEmailAndPassword()</code> + <code>Store.createUserProfile()</code>
admin-login.html	Admin	Admin gate: email + Master Key activation, calls <code>Store.setAdminRole()</code> if key matches
admin-dashboard.html	Admin	Executive dashboard: live revenue, total orders, pending count, recent orders table (aggregated from Firestore)
admin-products.html	Admin	Product CRUD: Add/Edit/Delete with modal form; image URL, pricing, stock, category, sizes, colors
admin-orders.html	Admin	Order management: filter by status, view full addresses, transition fulfillment stages, view cancellation logs

FILE	ROLE	KEY RESPONSIBILITIES
admin-coupons.html	Admin	Coupon manager: create % or flat discount codes, set min order threshold, activate/deactivate toggles
js/firebase-config.js	Core JS	Firebase SDK init with project config (<code>nari-niketan</code>); exports <code>auth</code> and <code>db</code> globals; sets auth persistence to LOCAL
js/store.js	Core JS	Centralized Firestore DAO: all CRUD operations, query builders, error handling, coupon lookup, order lifecycle methods
js/auth.js	Core JS	Authentication controller: Email, Google OAuth, Phone OTP methods; admin role verification helpers
js/cart.js	Core JS	Cart engine: <code>nn_cart</code> / <code>nn_coupon</code> localStorage keys, subtotal/GST/discount computation, badge update, coupon validator (Firestore + built-in)
js/products.js	Core JS	Catalog renderer: builds product card HTML, implements multi-criteria client-side filter, sorting algorithms
js/app.js	Core JS	Application lifecycle: <code>auth.onAuthStateChanged()</code> listener, navbar state, cart badge counter, toast system, scroll effects
js/admin.js	Core JS	Admin controllers: product modal CRUD handlers, order status transition buttons, coupon form actions, revenue aggregation
js/admin-guard.js	Core JS	Client-side RBAC route guard: verifies <code>isAdmin === true</code> before rendering any admin page; redirects to login if not authorised
css/style.css	Styles	Global design system: Burgundy/Gold CSS variables, Cinzel + Inter typography, hero, navbar, animations, responsive layout
css/shop.css	Styles	Product cards, filter sidebar, image galleries, cart drawer, order card layouts
css/auth.css	Styles	Auth form cards, OTP input grid, mode switcher, reCAPTCHA container
css/admin.css	Styles	Admin sidebar, metric stat cards, data tables, modal overlays, fulfillment status pill badges
firestore.rules	Security	Firestore server-side security rules: RBAC, data ownership checks, anti-elevation protection

FILE	ROLE	KEY RESPONSIBILITIES
firebase.json	Config	Firestore Hosting config: public directory, cache headers, SPA rewrites, rules file reference

9. HOSTING, CLOUD CONFIGURATION & DEPLOYMENT RUNBOOK

9.1 Current Hosting: Firebase Hosting (Active)

PROPERTY	VALUE
Firebase Project ID	nari-niketan
Firebase Auth Domain	nari-niketan.firebaseio.com
Storage Bucket	nari-niketan.firebaseiostorage.app
Live URLs	https://nari-niketan.web.app & https://nari-niketan.firebaseio.com
Hosting CDN	Firebase Edge CDN (30+ global PoPs, Brotli compression, DDoS protection)
SSL Certificate	Auto-provisioned & auto-renewing via Firebase (Let's Encrypt)
Caching Strategy	Cache-Control: no-cache, no-store, must-revalidate (always fresh, configured in firebase.json)
Deployed Files	All HTML, CSS, JS, images (PDF and dot-files excluded per firebase.json ignore rules)

9.2 Deploy Commands

```
# Install Firebase CLI (one time)
npm install -g firebase-tools

# Authenticate with Google account
firebase login

# Link to project (already configured in .firebaserc)
firebase use nari-niketan

# Deploy everything: hosting + Firestore security rules atomically
firebase deploy

# Deploy only hosting files (faster)
firebase deploy --only hosting
```

```
# Deploy only Firestore security rules
firebase deploy --only firestore:rules
```

9.3 Alternative: AWS S3 + CloudFront CDN

```
# Create S3 bucket in Mumbai region (ap-south-1 = lowest latency for India)
aws s3 mb s3://nariniketan-production --region ap-south-1
aws s3 website s3://nariniketan-production --index-document index.html --error-docume

# Upload all static files (excluding .git, firebase config, and PDFs)
aws s3 sync . s3://nariniketan-production \
  --exclude ".git/*" --exclude ".firebase/*" \
  --exclude "*.pdf" --exclude "firestore.rules"

# Create CloudFront CDN distribution with SSL (ACM certificate)
aws cloudfront create-distribution \
  --origin-domain-name nariniketan-production.s3-website.ap-south-1.amazonaws.com

# Point Route 53 custom domain (nariniketan.in) Alias A-record → CloudFront distribut
```

10. IMPLEMENTATION PLANNING & FEATURE PHASES

10.1 Architecture Decision Record

DECISION	CHOSEN	REJECTED	REASON
Frontend Framework	Vanilla HTML5/ES6	React, Vue, Next.js	0ms hydration, mobile-first, no build pipeline needed
Database	Cloud Firestore	PostgreSQL, MongoDB	Direct browser access, declarative security rules, serverless
Authentication	Firebase Auth	Custom JWT, Auth0	3 providers in 1 SDK, auto-integrates with Firestore rules, free tier
Cart Storage	LocalStorage	Redis, DB sessions	0ms latency, guest support, zero server cost
Hosting	Firebase Hosting	EC2, cPanel, Netlify	Atomic deploy with Firestore rules, global CDN, zero Linux admin
CSS Approach	Bespoke Vanilla CSS3	Tailwind, Bootstrap	Custom Indian aesthetic, no unused CSS, exact brand control
Security Model	Firestore Server Rules	API gateway auth	Rules evaluated server-side by Google, cannot be bypassed by client

10.2 Phase Implementation Breakdown

PHASE	FEATURES DELIVERED	STATUS
Phase 1 — Foundation	Firebase project setup, HTML/CSS design system (Burgundy & Gold), <code>firebase-config.js</code> , <code>store.js</code> DAO, <code>app.js</code> lifecycle, homepage, Firestore connection	✓ Complete
Phase 2 — Catalog	<code>shop.html</code> with multi-criteria filtering, <code>product.html</code> gallery & variant selection, <code>products.js</code> renderer, real-time stock indicators	✓ Complete
Phase 3 — Auth	<code>login.html</code> + <code>register.html</code> , Firebase Auth (Email/Google/Phone OTP), reCAPTCHA, <code>auth.js</code> , user profile creation in Firestore	✓ Complete
Phase 4 — Cart & Checkout	<code>cart.js</code> LocalStorage engine, <code>cart.html</code> drawer, <code>checkout.html</code> address + payment, <code>Store.addOrder()</code> , <code>confirmation.html</code> receipt	✓ Complete
Phase 5 — Admin Suite	Admin Portal (login gate, dashboard, product CRUD, order fulfillment table), <code>admin-guard.js</code> RBAC, <code>admin.js</code> , Master Key activation	✓ Complete
Phase 6 — Coupons & Cancellation	Dynamic Firestore coupon engine, <code>admin-coupons.html</code> , 1-Click Order Cancellation modal with reason selector in <code>my-orders.html</code>	✓ Complete
Phase 7 — Polish & Deploy	Firestore security rules hardening, Firebase Hosting deploy, <code>firebase.json</code> cache headers, responsive mobile QA, cross-browser testing	✓ Complete

10.3 Recommended Future Enhancements

- **Razorpay / PhonePe Integration** — Replace the COD/UPI static options with a live payment gateway for real INR transaction processing
- **Firebase Cloud Messaging (FCM)** — Push notifications to customers when order status changes (Shipped / Out for Delivery)
- **Algolia Search** — Instant full-text search across 10,000+ products with typo-tolerance, replacing client-side filter logic
- **Firebase Cloud Functions** — Server-side order confirmation emails (via SendGrid), inventory deduction triggers, and revenue report generation
- **PWA (Progressive Web App)** — Add `manifest.json` + Service Worker for offline catalog browsing and "Add to Home Screen" on mobile